

# Ponytail: make your AI write less code

Everything about Ponytail in one page: what it is, how it works, honest numbers, install for every tool, the CLAUDE.md rule if you can't install plugins, and answers to common questions.

---

## START HERE

### What is Ponytail?

**Ponytail** is a free, open-source (MIT) skill for AI coding agents, made by [DietrichGebert](#). It makes your AI think like "the laziest senior dev in the room": before writing code, it checks whether the code is even needed, and whether something that already exists can do the job.

- **145K+ GitHub stars** (created June 2026), #1 GitHub trending repo of the day, week and month.
- Works with **Claude Code, Codex, GitHub Copilot, Cursor**, Gemini CLI, OpenCode, Windsurf, Cline and ~20 agents in total.
- Its rule: *"The best code is the code you never wrote."* But lazy, not careless: it never cuts validation, error handling, security or accessibility.

## THE IDEA

### How it works: the 7-step ladder

Before writing code, the agent stops at the **first** step that solves the task:

1. Does this need to exist? → no: skip it (YAGNI)
2. Already in this codebase? → reuse it, don't rewrite
3. Stdlib does it? → use it
4. Native platform feature? → use it
5. Installed dependency? → use it
6. One line? → one line
7. Only then: the minimum that works

It reads the code it is changing first, then picks the rung. **Lazy about the solution, never about understanding the problem.**

---

REAL EXAMPLES

## What normal AI writes vs what Ponytail picks

| TASK                   | NORMAL AI                                                   | WITH PONYTAIL                                                                                       |
|------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Date picker            | installs a library, writes a wrapper component + stylesheet | <code>&lt;input type="date"&gt;</code>                                                              |
| Colour picker          | custom component (287 lines in the benchmark)               | <code>&lt;input type="color"&gt;</code>                                                             |
| Deep copy an object    | npm install lodash → cloneDeep                              | <code>const copy = structuredClone(original);</code>                                                |
| Show ₹12,34,567        | npm install numeral                                         | <code>new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR" }).format(1234567)</code> |
| Group a list by status | lodash groupBy or a hand-written reduce                     | <code>Object.groupBy(orders, o =&gt; o.status)</code>                                               |
| Read URL parameters    | npm install query-string                                    | <code>new URLSearchParams(location.search).get("page")</code>                                       |

Examples from the Ponytail repo's `examples/` folder; the ₹ (en-IN) version is ours.

HONEST NUMBERS

## Does it really cut code?

From the **creator's own benchmark**: real Claude Code sessions on a FastAPI + React repo, 12 feature tasks, Claude Haiku 4.5, 4 runs each, compared with the same agent with no skill.

|          | CODE (LINES) | TOKENS | COST | TIME | SAFETY TESTS |
|----------|--------------|--------|------|------|--------------|
| Ponytail | -54%         | -22%   | -20% | -27% | 100% passed  |

- **Big wins** where AI over-builds: date picker 404 → 23 lines, colour picker 287 → 23, file dropzone 251 → 95.
- **Almost no change** where the code is already minimal (simple backend endpoints).
- The README says on some models (e.g. GPT-5.5) cost can go **up**, because the model spends extra thinking on the ladder.
- **The story**: the first claim was "80–94% less code". Colin Eberhardt (CTO, Scott Logic) showed that benchmark was unfair. The creator re-tested properly and published the lower, honest 54%.

INSTALL

**Install it in your tool**

## Claude Code

Send these as **two separate prompts** inside Claude Code (also works in the Desktop app's Code tab):

```
/plugin marketplace add DietrichGebert/ponytail
```

```
/plugin install ponytail@ponytail
```

Needs `node` on your PATH for its two small hooks.

## Codex (CLI + desktop app)

```
codex plugin marketplace add DietrichGebert/ponytail  
codex plugin add ponytail@ponytail
```

Then run `codex`, open `/hooks`, trust its two hooks, start a new thread.

## GitHub Copilot CLI

```
copilot plugin marketplace add DietrichGebert/ponytail  
copilot plugin install ponytail@ponytail
```

## Cursor

```
git clone https://github.com/DietrichGebert/ponytail  
node ponytail/scripts/cursor-hooks.js install
```

Or the rule-only way: copy `.cursor/rules/` from the repo into your project.

## Windsurf, Cline, Copilot Chat (VS Code / JetBrains), Kiro, Zed, Aider...

Copy the matching rules file from the repo ( `.windsurf/rules/` , `.clinerules/` , `.github/copilot-instructions.md` , `AGENTS.md` , `.kiro/steering/` ). VS Code's Codex extension, Amp and Jules read `AGENTS.md` automatically.

All tools: [github.com/DietrichGebert/ponytail#install](https://github.com/DietrichGebert/ponytail#install)

### USE IT

## Commands

| COMMAND                                          | WHAT IT DOES                                                           |
|--------------------------------------------------|------------------------------------------------------------------------|
| <code>/ponytail lite   full   ultra   off</code> | Set how strict it is, or turn it off. Default is <b>full</b> .         |
| <code>/ponytail-review</code>                    | Reviews your current diff and gives a delete-list of over-engineering. |
| <code>/ponytail-audit</code>                     | Checks the whole repo for over-engineering.                            |
| <code>/ponytail-debt</code>                      | Collects the shortcuts it deferred, so 'later' doesn't become 'never'. |
| <code>/ponytail-help</code>                      | Quick reference.                                                       |

Commands work in Claude Code, Codex (use `@ponytail-review` ), Gemini, OpenCode and other skill-capable tools. Rule-file installs (Windsurf, Cline, Copilot Chat) get the rule without commands.

### COMPANY BLOCKS OUTSIDE PLUGINS?

## Paste the rule instead (no code, nothing to install)

Put this in `CLAUDE.md` at your project root (or `~/ .claude/CLAUDE.md` for all projects). For Codex, Cursor, Copilot, Amp, Jules: save the same text as `AGENTS.md`.

```
## Write the minimum code (lazy senior dev rule)
```

Before writing any code, stop at the FIRST step that solves the task:

1. Does this need to exist? If not, skip it and say why.
2. Is it already in this codebase? Reuse it, don't rewrite it.
3. Does the language's standard library do it? Use that.
4. Does the platform do it natively (`<input type="date">`, `Intl.NumberFormat`, `structuredClone`)? Use that.
5. Does an already-installed dependency do it? Use that. Never add a new package for one function.
6. Can it be one line? Write one line.
7. Only then: write the minimum that works.

Read the code you are changing first. Be lazy about the solution, never about understanding the problem.

Never cut: input validation at trust boundaries, error handling that prevents data loss, security, accessibility.

## Or as a Claude skill

Save as `.claude/skills/lazy-senior/SKILL.md` in your project:

```
---
```

```
name: lazy-senior
```

```
description: Use before writing or changing code. Makes you pick the smallest correct solution (reuse, stdlib, native feature, installed dependency) before writing new code, without dropping validation, error handling, security or accessibility.
```

```
---
```

```
## Write the minimum code (lazy senior dev rule)
```

Before writing any code, stop at the FIRST step that solves the task:

1. Does this need to exist? If not, skip it and say why.
2. Is it already in this codebase? Reuse it, don't rewrite it.
3. Does the language's standard library do it? Use that.

4. Does the platform do it natively (`<input type="date">`, `Intl.NumberFormat`, `structuredClone`)? Use that.
5. Does an already-installed dependency do it? Use that. Never add a new package for one function.
6. Can it be one line? Write one line.
7. Only then: write the minimum that works.

Read the code you are changing first. Be lazy about the solution, never about understanding the problem.

Never cut: input validation at trust boundaries, error handling that prevents data loss, security, accessibility.

This is a lighter version: the plugin also runs hooks that keep it active every session and adds the commands. The creator's own data shows a short pasted prompt works less consistently than the full skill, so review what your AI writes. Rule text adapted from Ponytail's ladder (MIT, © DietrichGebert).

REMOVE IT

## Uninstall

| TOOL                                         | COMMAND                                             |
|----------------------------------------------|-----------------------------------------------------|
| Claude Code                                  | <code>/plugin remove ponytail</code>                |
| Codex                                        | <code>codex plugin remove ponytail</code>           |
| Cursor hooks                                 | <code>node scripts/cursor-hooks.js uninstall</code> |
| Rule files (Cursor rule, Windsurf, Cline...) | Delete the copied file                              |

To also clean its small settings files, run `node scripts/uninstall.js` from the repo **before** removing the plugin.

QUESTIONS PEOPLE ASK





### **Is it free?**

Yes. MIT licence, no account, no API key. Ponytail itself costs nothing; your AI tool's plan stays the same.

### **Is it safe to install?**

It's open source, so you can read every line. The Claude Code / Codex plugin runs two tiny Node.js hooks that add the rule to each session. If your company doesn't allow that, use the paste-the-rule option above: it's plain text, no code runs.

### **Will it break or delete my existing code?**

No. It changes how the AI writes **new** code. It doesn't touch files on its own. Always review the diff like you normally would.

### **Does it really save tokens and money?**

In the creator's benchmark: about 22% fewer tokens and 20% lower cost on Claude Haiku 4.5. On other models it varies, and on some it can cost more. Test it on your own project.

### **My project uses a design system (shadcn, MUI...). Will it ignore it?**

Step 2 of the ladder is 'already in this codebase? reuse it', which comes before native browser features, so it should reuse your components. This case isn't in the benchmark, so check its first few changes.

### **Does it make the AI lazy about security?**

No. The rule says validation, data-loss handling, security and accessibility are never cut. In the creator's safety tests it passed 100%; a plain 'write one-liners' prompt dropped one guard.

### Can I turn it off for one task?

Yes: `/ponytail off`, or `/ponytail lite` for a softer version.

### Does it work with caveman (the short-answers skill)?

Yes, the README recommends it. Caveman shortens what the AI says; Ponytail shortens what it builds.

### Why 'Ponytail'?

Every office has that one senior dev with a ponytail who replaces your 50 lines with one. Ponytail puts him inside your AI.

## SOURCES

### Links

- [Ponytail on GitHub](#)
- [Creator's benchmark write-up](#)
- [The critique: "Ponytail? YAGNI!" \(Scott Logic\)](#)
- [InfoQ: Ponytail corrects its own benchmark](#)

Guide by @shashikant.ai · facts checked 24 Sep 2026. Stars and features change; the GitHub repo is the source of truth.